

Difference between Functions, Triggers & Stored Procedures

1. Functions (CREATE FUNCTION)

✓ Definition:

A function is a reusable SQL or PL/pgSQL code block that **returns a value** and can be used in queries, expressions, or procedural logic.

✓ When to Use:

- When you want to return a computed value (e.g., tax calculation, string manipulation).
- For encapsulating reusable logic inside SELECT/UPDATE/INSERT statements.
- When you need to create **triggers** (triggers in PostgreSQL require a function as a handler).

✓ Types:

- **Scalar functions** – return a single value.
- **Table-returning functions (set-returning)** – return a result set like a SELECT.
- **Trigger functions** – used **only inside a trigger** (RETURNS TRIGGER).

✓ Capabilities:

- Can return values.
- Can be used in SELECT statements.
- Can contain control structures (IF, LOOP, etc.).
- Can call other functions.
- Can be deterministic (same input → same output).

✗ Limitations:

- Cannot perform transactional control (COMMIT, ROLLBACK).
- Cannot execute CALL (only SELECT function() or as part of a query).
- Cannot perform out-of-band operations (like raising notices without returning).

◆ 2. Stored Procedures (CREATE PROCEDURE)

✓ Definition:

Stored procedures are blocks of SQL/PL code that **do not return a value**, but **can perform complex logic**, including transactional control.

✓ When to Use:

- When you need to **control transactions explicitly** (COMMIT, ROLLBACK).
- For **batch operations**, maintenance tasks, or administrative workflows.
- For **multi-step logic** that needs rollback on failure.

✓ Types:

- Standard procedure with CALL.
- Can be written in PL/pgSQL, SQL, or other supported languages.

✓ Capabilities:

- Can contain full transaction control.
- Can use CALL to execute.
- Can raise notices, exceptions, perform complex logic.
- Can call other procedures/functions.

✗ Limitations:

- Cannot be used in SELECT queries.
- Cannot be used directly as triggers.
- Cannot return values directly (but can use OUT parameters).

◆ 3. Triggers (CREATE TRIGGER)

✓ Definition:

A trigger is a special object that **automatically executes a function** in response to a **data modification event** (e.g., INSERT, UPDATE, DELETE).

✓ When to Use:

- For enforcing business rules (e.g., preventing certain deletes).
- Automatically updating audit logs or timestamps.
- Enforcing complex integrity constraints not possible with CHECK.

✓ Types:

- **BEFORE** Trigger – runs before the event.
- **AFTER** Trigger – runs after the event.
- **INSTEAD OF** Trigger – used on views.
- **ROW-level** or **STATEMENT-level**.

✅ Capabilities:

- Automatically invoked.
- Can modify NEW or OLD records (ROW-level).
- Can stop or allow operations (RETURN NULL cancels insert/update).
- Can log, validate, or cascade actions.

❌ Limitations:

- Must call a RETURNS TRIGGER function.
- Cannot use transaction control (no COMMIT, ROLLBACK).
- Difficult to debug; should be used cautiously to avoid performance issues.

◆ BEFORE Triggers

✅ When to Use:

Use BEFORE triggers when you want to **modify or validate the data before it is written** to the table.

✅ Typical Use Cases:

- **Data validation:** Prevent invalid data from being inserted or updated.
- **Auto-filling/modifying fields:** E.g., auto-formatting strings, setting default values.
- **Cancelling operations:** You can return NULL in a BEFORE INSERT/UPDATE to cancel the action.
- **Audit control:** Enforcing constraints that go beyond CHECK.

❌ Limitations:

- You **can't access auto-generated values** like sequences or timestamps (they haven't been assigned yet).
- You can't log changes yet because the change isn't finalized.

◆ AFTER Triggers

✅ When to Use:

Use AFTER triggers when you need to **take action based on the final state of the row**, such as **logging, cascading changes, or updating related tables**.

✅ Typical Use Cases:

- **Audit logging:** Recording who did what and when.

- **Cascading changes:** Updating related data in other tables (e.g., denormalized values).
- **Sending notifications:** Like calling external systems or queues.
- **Archiving deleted data.**

✗ Limitations:

- You **cannot modify the row itself** (use BEFORE if you want to change NEW).
- If the original operation fails (e.g., due to constraints), the AFTER trigger won't fire.

◆ INSTEAD OF Triggers

✓ When to Use:

Use INSTEAD OF triggers when you want to **intercept and replace** the action of an INSERT, UPDATE, or DELETE on a **view**, and define **custom behavior** for it.

✓ Typical Use Cases:

- **Making views updatable:** Allow INSERT, UPDATE, or DELETE on complex views that aren't natively updatable.
- **Redirecting changes:** Route DML operations from a view to one or more underlying base tables.
- **Simulating virtual tables:** Create a view that behaves like a table but whose logic is fully implemented in triggers.
- **Controlling write access:** Enforce business logic at the view level without exposing base tables.

✗ Limitations:

- Can only be defined on **views**, not on base tables.
- Only **one trigger per operation type** (INSERT, UPDATE, or DELETE) per view is allowed.
- You **must handle the entire logic yourself** inside the trigger (e.g., validating data, managing constraints).
- Cannot use BEFORE or AFTER triggers on views—**only INSTEAD OF is supported for views.**
- No automatic support for things like RETURNING clauses—you must explicitly handle return values if needed.

◆ **Comparison of all types of Triggers**

Feature	BEFORE	AFTER	INSTEAD OF
Applies To	Tables	Tables	Views only
Runs Before/After	Before event	After event	Instead of the event
Can Modify Data	✔ Yes	✗ No	✔ Yes (but must fully handle)
Use Case	Validation, changes	Logging, cascade	Custom view logic
 Cancels Operation?	✔ (return NULL)	✗ (too late)	✔ (entire operation replaced)